

Tiny Tapeout Chipdesign Workshop @KIT

Design a chip and have it manufactured on an open source ASIC process!



You will learn

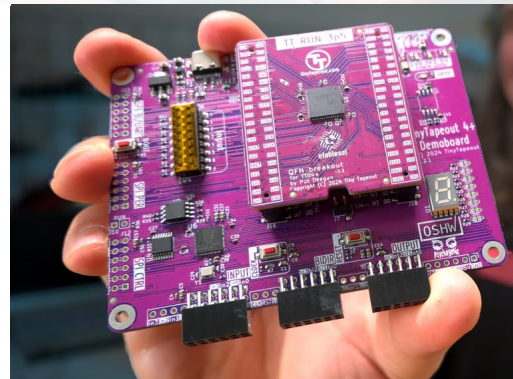
- 🔧 Semiconductors
How chips are made
- 🔌 Logic
The basics of digital circuits
- 💻 Simulation
Test your own design
- 🏭 Manufacturing
Create GDS files for IHP

When and where

- 🕒 Sat., 07.02.2026, 10:00 – 15:00
Registration required!
- 📍 NTI Lecture Hall, Bld. 30.10 @KIT
- 👤 All students (bachelor, master, PhD)
- 🔑 Bring a laptop and your GitHub account
- 🔑 **Costs (incl. lunch) covered by KCH!**

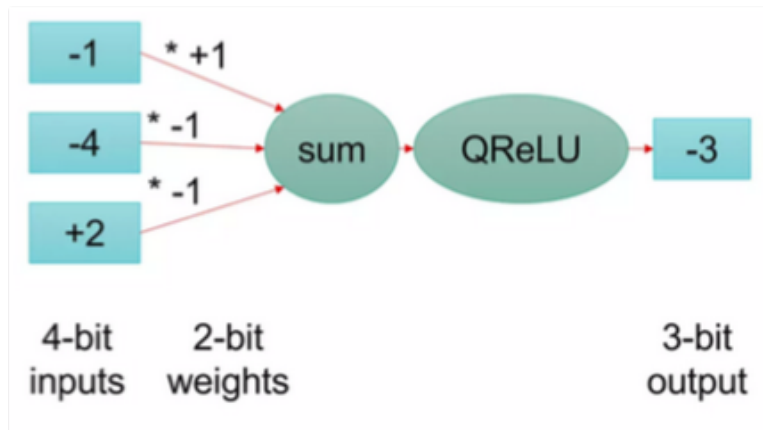


Register at kch.kit.edu/tt



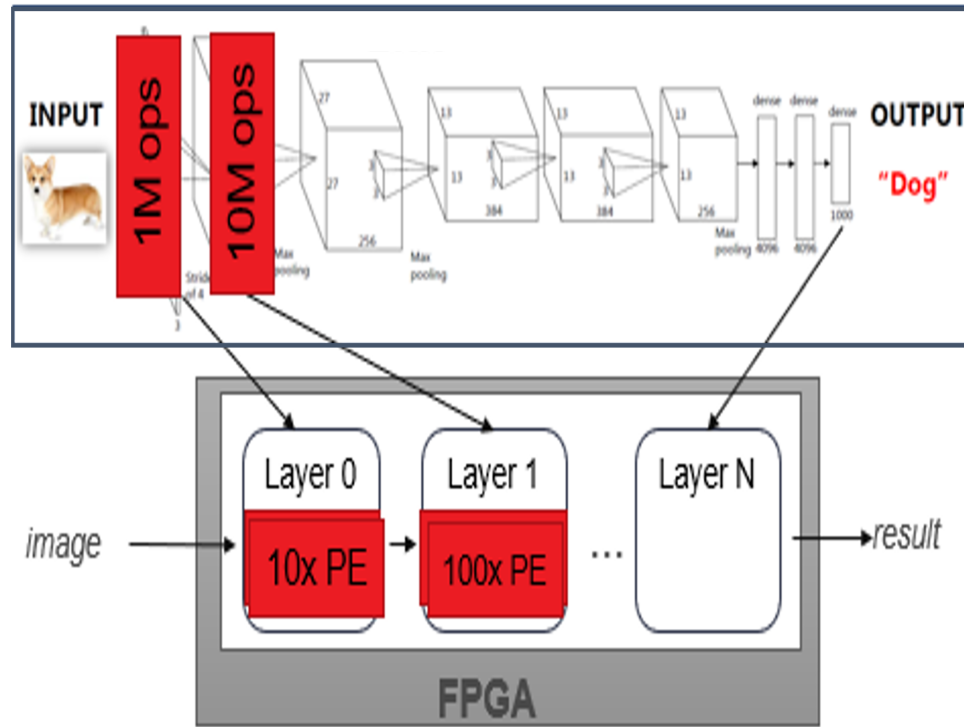
Project: Neural Network Inference on FPGA

- NN training using **Brevitas** ([Tutorial](#)) and **Pytorch**
 - Brevitas -- 0.11.0, Python -- 3.10.19, Pytorch -- 2.4.1, Torchvision -- 0.19.1
- Quantization Aware Training (QAT) and Post-Training Quantization (PTQ)
 - Provided template training script uses QAT
 - Forward pass includes fake quantization (clamp, round, scale to low bit, e.g. int8, int4, binary)
 - Backward pass uses straight-through estimator so that gradients can flow

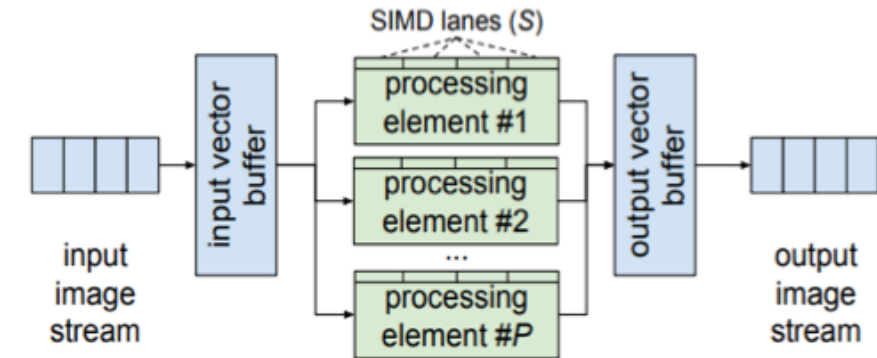


Trillions of quantized operations per second
and weights can stay entirely on-chip

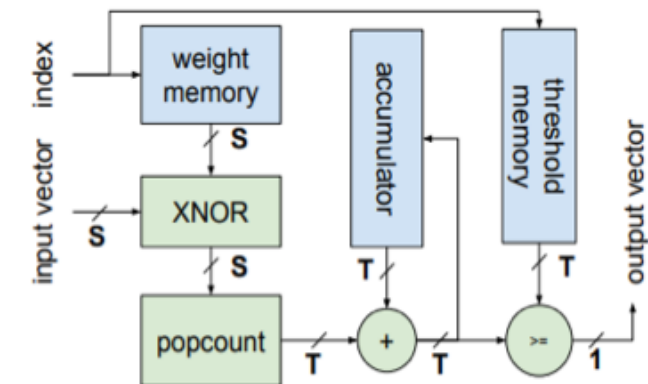
FINN Framework_[2] dataflow architecture



End-to-end flow for quantized neural network inference



Layer of NN is implemented as MVAU Block



PE (Processing Element) architecture

- Template training script for a MLP model is given as starting point
 - Define and train you own NN model.
 - Calculate the accuracy of your trained model with test dataset.
 - Save your model state and trained parameters in “.pth” format.
- Basic Finn Framework workflow scripts are also provided
 - Follow the instructions ([FINN tutorial](#)) to set up your own FINN work environment.
 - Environment variables:
 - `export FINN_XILINX_PATH=/Software/Xilinx`
 - `export FINN_XILINX_VERSION=2022.2`
 - `export FINN_HOST_BUILD_DIR=”create a folder for generated vivado project files (absolute path)”`
 - Modify “folding_config.json” and “specialize_layers_config.json” based on your model architecture
 - Intermediate models (ONNX format) of each finn workflow step can be visualized with [netron](#)

Tasks

- Under “/finn_flow/output_bitfile/report/” check “rtlsim_performance.json” to see conclusion of hardware implementation performance and modify PE/SIMD to optimize it.
- Under “/finn_flow/output_bitfile/” see “deploy” folder and upload entire folder to your PYNQ
 - Run inference for single input: **python3 driver.py --exec_mode=execute --batchsize=1 --bitfile=yourfile.bit --input_file=input.npy**
 - Validate the accuracy: **python3 validate.py --bitfile=yourfile.bit --dataset mnist --batchsize 1000**
 - In case you can not download dataset via “--dataset”, you need to upload dataset manually and use “--dataset_root (your file path)”
- For evaluation a short report is expected that describes your NN model and source files (scripts, reports, bitfiles)